Introduction to Parallel Algorithms and Programming

Session 6: Parallel algorithms on communication networks

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Add the **cost of communication** to our model of a parallel machine: distributed memory, links, latency and bandwidth
- Explore the classical **network topologies** and the parameters that characterize them
- Learn the basic **communication algorithms** on a ring (broadcast, reduction), and analyse their cost
- Design first **algorithms on a ring** with these routines, a 1-D convolution and the normalization of a vector, and analyse their cost
- Keep the principle of the course: every processor runs the **same program** on its own index; the processors now exchange data by **messages** instead of a shared memory

Outline

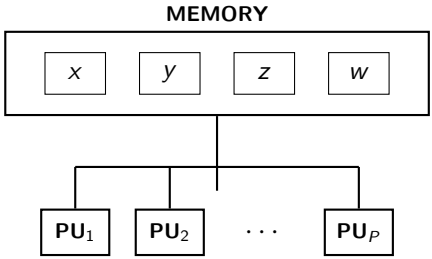
1. Distributed memory and communication networks
2. Network parameters and cost
3. Communication on a ring
4. Algorithms on a ring

Outline

1. Distributed memory and communication networks
2. Network parameters and cost
3. Communication on a ring
4. Algorithms on a ring

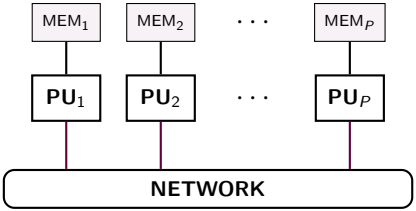
Communication in the PRAM

- In the PRAM, the PUs are connected by the **shared memory**.
- To exchange data, a PU writes a cell and another PU reads it.
- Every cell is accessed in constant time: the cost of a communication is $O(1)$, **the same for every pair** of PUs.
- Convenient for designing algorithms, but **no real parallel machine works like this**: a memory that P processors access in constant time cannot be built, due to physical constraints, once P is large.
- Example: a GPU **SM** behaves roughly like a PRAM with $P = 32$ synchronous threads and constant-time access to its shared memory; beyond that, a GPU uses many SMs, which are **independent and asynchronous**.



A real parallel machine: distributed memory

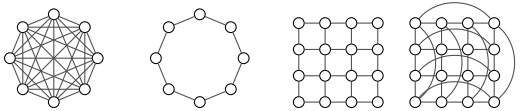
- The memory is **distributed**: each processor has its own **local memory**, which the other processors cannot access.
- The processors are connected by **links**, which form the **communication network**.
- Data are exchanged by **explicit communications** over the network: a processor **sends** a message, another one **receives** it.
- The cost of a communication is no longer constant: it depends on the **position** of the two processors in the network, and on the **parameters** of the network: topology, latency, bandwidth.
- The model of this session, and of MPI (session 7).





Network topology

- The graph of the links between the processors is the **topology** of the network:
  - each node is a **processor**;
  - each edge is a **communication link** between two processors.
- **Static network**: the links are fixed once and for all. Examples: (a) clique, (b) ring, (c) 2-D grid, (d) 2-D torus, (e) hypercube.
- **Dynamic network**: the links are configured at run time by switches. Examples: (f) fat-tree, (g) butterfly.
- More links: faster communication, but a more expensive network. Every topology is a **trade-off** between cost and performance.



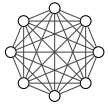
Outline

1. Distributed memory and communication networks
2. Network parameters and cost
3. Communication on a ring
4. Algorithms on a ring

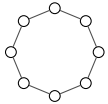
Network parameters

A topology is characterized by a few parameters:

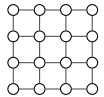
- **Number of processors** p: the nodes of the graph.
- **Degree** k: the number of links of a node (k_{min}, k_{max} when it varies). The hardware cost of a node.
- **Diameter** D: the maximum distance (number of links to cross) between two nodes. The worst-case cost of a communication.
- **Number of links** N_l: the hardware cost of the network.
- **Bisection width** L_B: the minimum number of links to remove to split the network into **two halves** ($p/2$ processors on each side). How much traffic can cross the network at once.



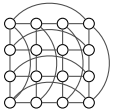
(a) clique



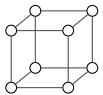
(b) ring



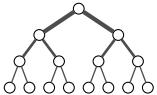
(c) 2-D grid



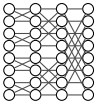
(d) 2-D torus



(e) hypercube



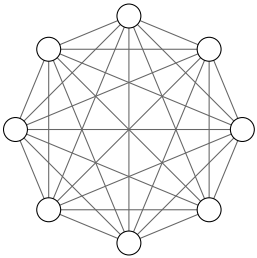
(f) fat-tree



(g) butterfly

The clique

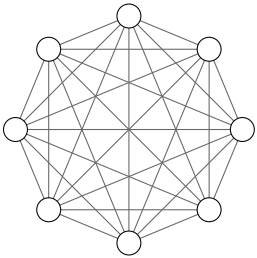
p processors; every pair of processors is linked: the complete graph.



The clique

p processors; every pair of processors is linked: the complete graph.

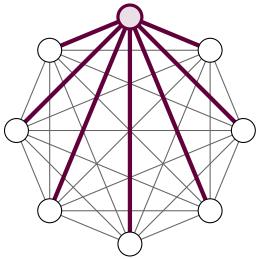
- **Degree** k?



The clique

p processors; every pair of processors is linked: the complete graph.

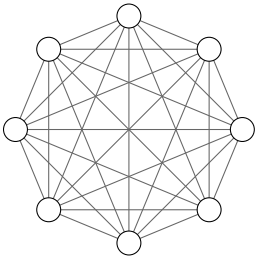
- **Degree** k? $p-1$: each node is linked to all the others.



The clique

p processors; every pair of processors is linked: the complete graph.

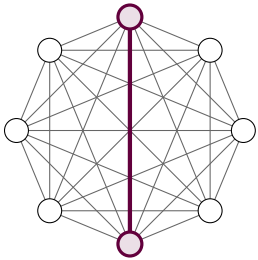
- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D?



The clique

p processors; every pair of processors is linked: the complete graph.

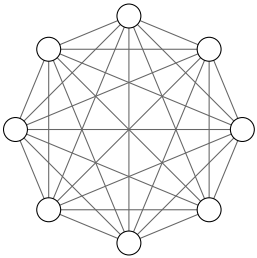
- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D? 1: every node is one link away.



The clique

p processors; every pair of processors is linked: the complete graph.

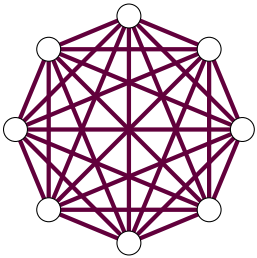
- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D? 1: every node is one link away.
- **Number of links** N_l?



The clique

p processors; every pair of processors is linked: the complete graph.

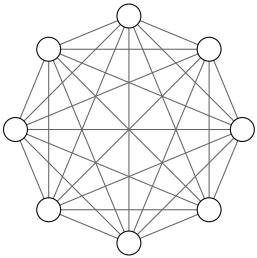
- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D? 1: every node is one link away.
- **Number of links** N_l? $p(p-1)/2$: p nodes with $p-1$ links each, every link counted twice.



The clique

p processors; every pair of processors is linked: the complete graph.

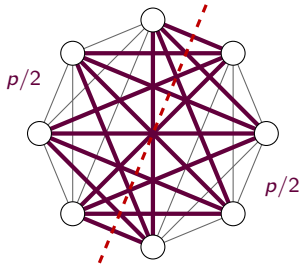
- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D? 1: every node is one link away.
- **Number of links** N_l? $p(p-1)/2$: p nodes with $p-1$ links each, every link counted twice.
- **Bisection width** L_B?



The clique

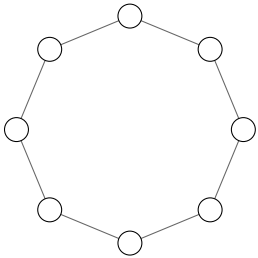
p processors; every pair of processors is linked: the complete graph.

- **Degree** k? $p-1$: each node is linked to all the others.
- **Diameter** D? 1: every node is one link away.
- **Number of links** N_l? $p(p-1)/2$: p nodes with $p-1$ links each, every link counted twice.
- **Bisection width** L_B? $(p/2)^2$: each of the $p/2$ nodes of one half is linked to the $p/2$ nodes of the other half.



The ring

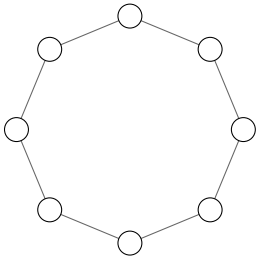
p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

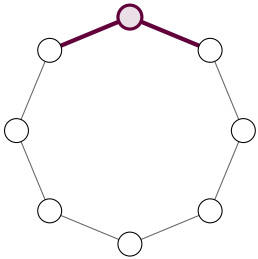
- **Degree** k?



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

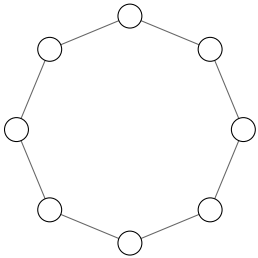
- **Degree** k? 2: the two neighbours.



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

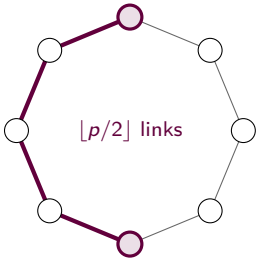
- **Degree** k? 2: the two neighbours.
- **Diameter** D?



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

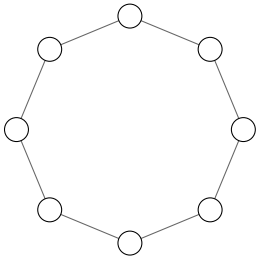
- **Degree** k? 2: the two neighbours.
- **Diameter** D? $\lfloor p/2 \rfloor$: to reach the opposite node, go round the ring the shorter way.



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

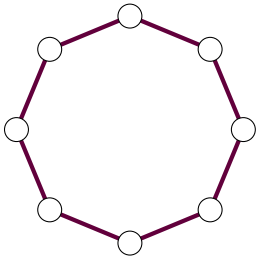
- **Degree** k? 2: the two neighbours.
- **Diameter** D? $\lfloor p/2 \rfloor$: to reach the opposite node, go round the ring the shorter way.
- **Number of links** N_l?



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

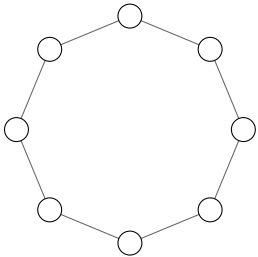
- **Degree** k? 2: the two neighbours.
- **Diameter** D? $\lfloor p/2 \rfloor$: to reach the opposite node, go round the ring the shorter way.
- **Number of links** N_l? p: one link per node, to its right neighbour.



The ring

p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

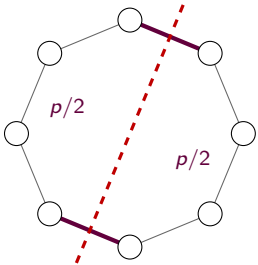
- **Degree** k? 2: the two neighbours.
- **Diameter** D? $\lfloor p/2 \rfloor$: to reach the opposite node, go round the ring the shorter way.
- **Number of links** N_l? p: one link per node, to its right neighbour.
- **Bisection width** L_B?



The ring

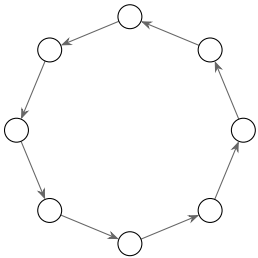
p processors $0, \ldots, p-1$; processor i is linked to its two neighbours $i-1$ and $i+1$ (modulo p).

- **Degree** k? 2: the two neighbours.
- **Diameter** D? $\lfloor p/2 \rfloor$: to reach the opposite node, go round the ring the shorter way.
- **Number of links** N_l? p: one link per node, to its right neighbour.
- **Bisection width** L_B? 2: cutting the ring into two arcs removes two links.



The unidirectional ring

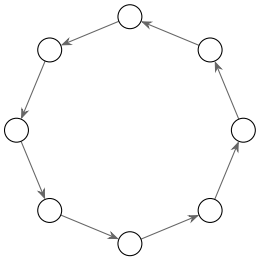
The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

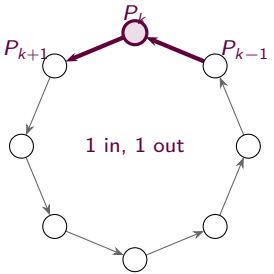
- **Degree** k?



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

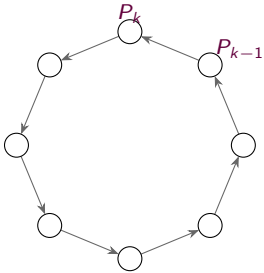
- **Degree** k? 2: one incoming link and one outgoing link per node.



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

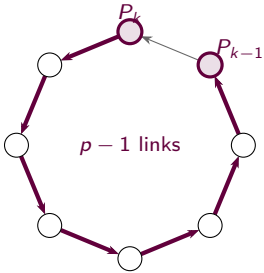
- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D?



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

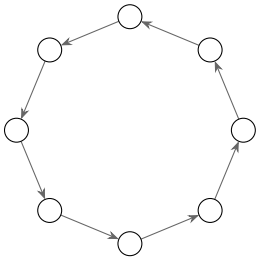
- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D? $p-1$: the farthest node is the left neighbour P_{k-1}: a message to it goes all the way round the ring.



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

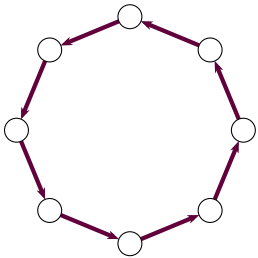
- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D? $p-1$: the farthest node is the left neighbour P_{k-1}: a message to it goes all the way round the ring.
- **Number of links** N_l?



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

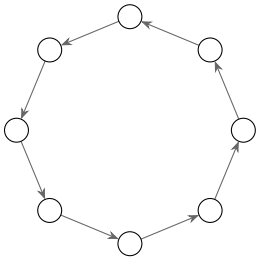
- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D? $p-1$: the farthest node is the left neighbour P_{k-1}: a message to it goes all the way round the ring.
- **Number of links** N_l? p: one outgoing link per node, as for the ring.



The unidirectional ring

The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

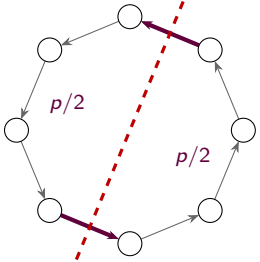
- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D? $p-1$: the farthest node is the left neighbour P_{k-1}: a message to it goes all the way round the ring.
- **Number of links** N_l? p: one outgoing link per node, as for the ring.
- **Bisection width** L_B?



The unidirectional ring

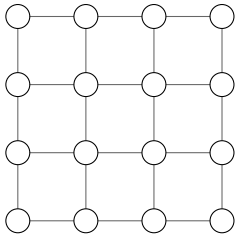
The ring of p processors with **one-way links**: the link between P_k and P_{k+1} carries messages in one direction only. P_k can **send to** P_{k+1}, and can only **receive from** P_{k-1} (indices modulo p).

- **Degree** k? 2: one incoming link and one outgoing link per node.
- **Diameter** D? $p-1$: the farthest node is the left neighbour P_{k-1}: a message to it goes all the way round the ring.
- **Number of links** N_l? p: one outgoing link per node, as for the ring.
- **Bisection width** L_B? 2: the two links between the two arcs, as for the ring; but only one of them enters each half.



The 2-D grid

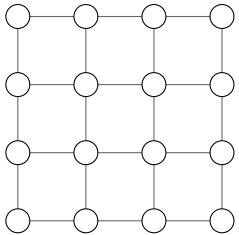
$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

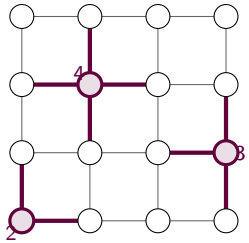
- **Degree** k?



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

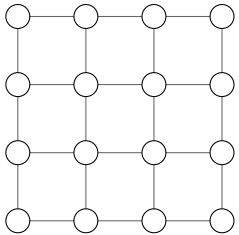
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

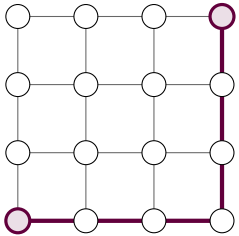
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D?



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

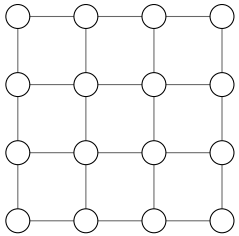
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D? $2(\sqrt{p} - 1)$: from a corner to the opposite one, $\sqrt{p} - 1$ links in each direction.



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

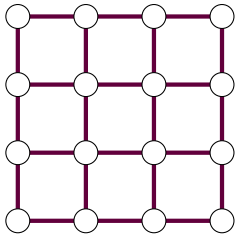
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D? $2(\sqrt{p} - 1)$: from a corner to the opposite one, $\sqrt{p} - 1$ links in each direction.
- **Number of links** N_l?



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

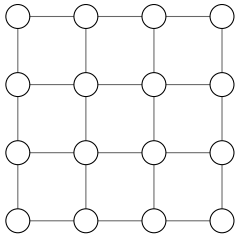
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D? $2(\sqrt{p} - 1)$: from a corner to the opposite one, $\sqrt{p} - 1$ links in each direction.
- **Number of links** N_l? $2p - 2\sqrt{p}$: $\sqrt{p}$ rows of $\sqrt{p} - 1$ links, and as many columns.



The 2-D grid

$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

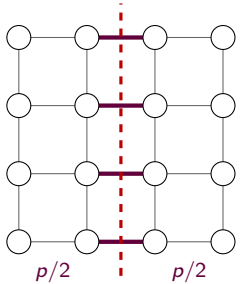
- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D? $2(\sqrt{p} - 1)$: from a corner to the opposite one, $\sqrt{p} - 1$ links in each direction.
- **Number of links** N_l? $2p - 2\sqrt{p}$: $\sqrt{p}$ rows of $\sqrt{p} - 1$ links, and as many columns.
- **Bisection width** L_B?



The 2-D grid

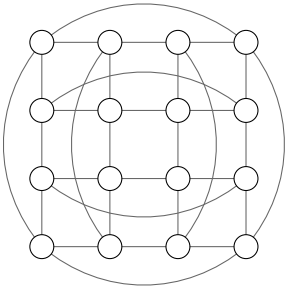
$p = \sqrt{p} \times \sqrt{p}$ processors at the positions (i, j); processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ when they exist.

- **Degree** k? 2 to 4: 2 at the corners, 3 on the sides, 4 inside.
- **Diameter** D? $2(\sqrt{p} - 1)$: from a corner to the opposite one, $\sqrt{p} - 1$ links in each direction.
- **Number of links** N_l? $2p - 2\sqrt{p}$: $\sqrt{p}$ rows of $\sqrt{p} - 1$ links, and as many columns.
- **Bisection width** L_B? $\sqrt{p}$: cut the $\sqrt{p}$ links between two middle columns.



The 2-D torus

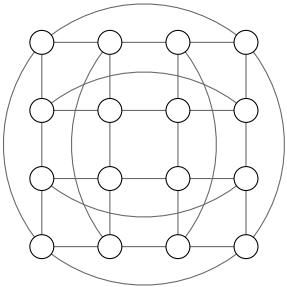
A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

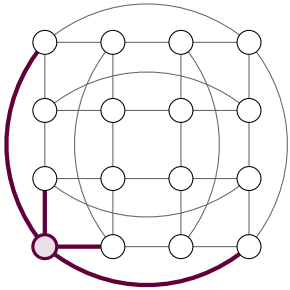
- **Degree** k?



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

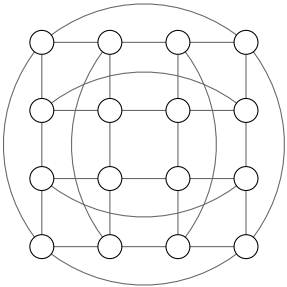
- **Degree** k? 4: everywhere, there is no border any more.



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

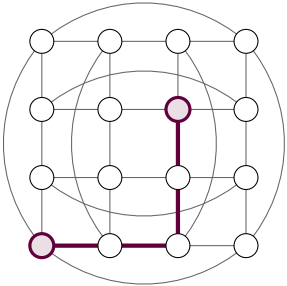
- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D?



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

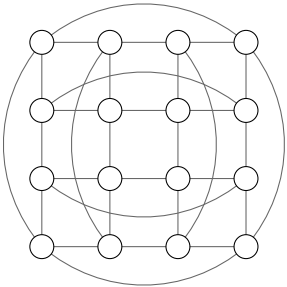
- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D? $2\lfloor \sqrt{p}/2 \rfloor$: $\lfloor \sqrt{p}/2 \rfloor$ in each direction, as on a ring of $\sqrt{p}$ nodes.



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

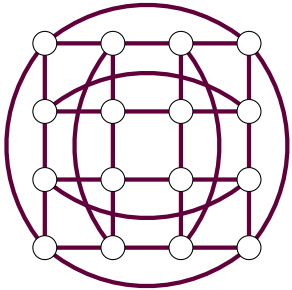
- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D? $2\lfloor \sqrt{p}/2 \rfloor$: $\lfloor \sqrt{p}/2 \rfloor$ in each direction, as on a ring of $\sqrt{p}$ nodes.
- **Number of links** N_l?



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

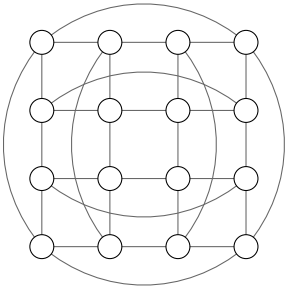
- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D? $2\lfloor \sqrt{p}/2 \rfloor$: $\lfloor \sqrt{p}/2 \rfloor$ in each direction, as on a ring of $\sqrt{p}$ nodes.
- **Number of links** N_l? $2p$: two links per node (to the right and upwards), none missing.



The 2-D torus

A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

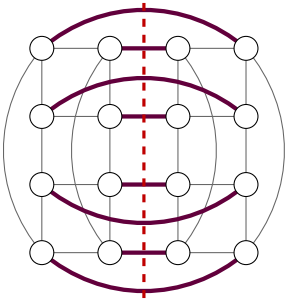
- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D? $2\lfloor\sqrt{p}/2\rfloor$: $\lfloor\sqrt{p}/2\rfloor$ in each direction, as on a ring of $\sqrt{p}$ nodes.
- **Number of links** N_l? $2p$: two links per node (to the right and upwards), none missing.
- **Bisection width** L_B?



The 2-D torus

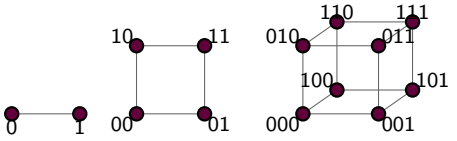
A $\sqrt{p} \times \sqrt{p}$ grid with wraparound links: processor (i, j) is linked to $(i \pm 1, j)$ and $(i, j \pm 1)$ modulo $\sqrt{p}$: every row and every column is a ring.

- **Degree** k? 4: everywhere, there is no border any more.
- **Diameter** D? $2\lfloor\sqrt{p}/2\rfloor$: $\lfloor\sqrt{p}/2\rfloor$ in each direction, as on a ring of $\sqrt{p}$ nodes.
- **Number of links** N_l? $2p$: two links per node (to the right and upwards), none missing.
- **Bisection width** L_B? $2\sqrt{p}$: a cut between two columns crosses $\sqrt{p}$ links, plus the $\sqrt{p}$ wraparound links.

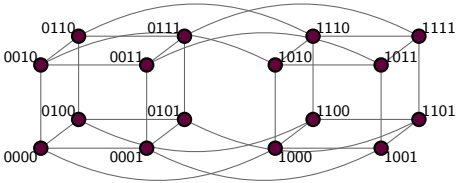


The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.



1-cube 2-cube 3-cube

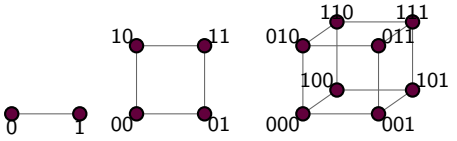


4-cube: two 3-cubes linked across dimension 3

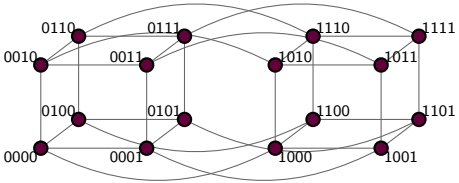
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k?



1-cube 2-cube 3-cube

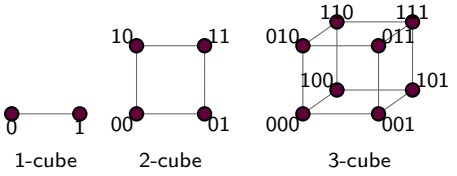


4-cube: two 3-cubes linked across dimension 3

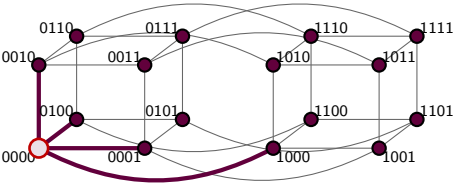
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.



1-cube 2-cube 3-cube

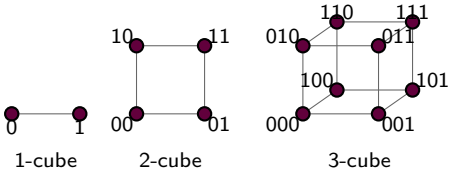


4-cube: two 3-cubes linked across dimension 3

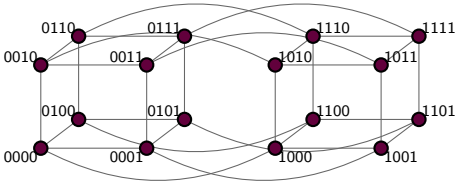
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D?



1-cube 2-cube 3-cube

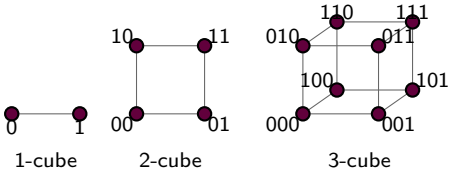


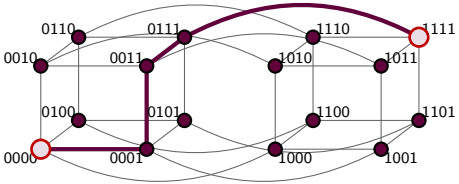
4-cube: two 3-cubes linked across dimension 3

The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D? $d = \log_2 p$: flip the differing bits one by one.



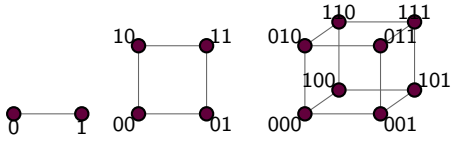


4-cube: two 3-cubes linked across dimension 3

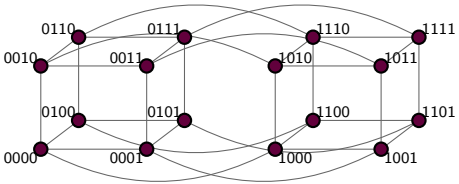
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D? $d = \log_2 p$: flip the differing bits one by one.
- **Number of links** N_l?



1-cube 2-cube 3-cube

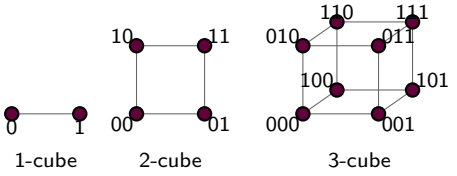


4-cube: two 3-cubes linked across dimension 3

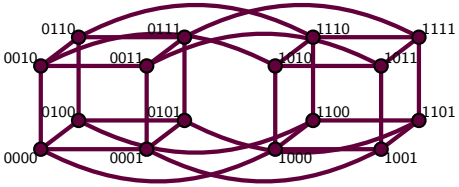
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D? $d = \log_2 p$: flip the differing bits one by one.
- **Number of links** N_l? $p \log_2 p/2$: p nodes with d links each, every link counted twice.



1-cube 2-cube 3-cube

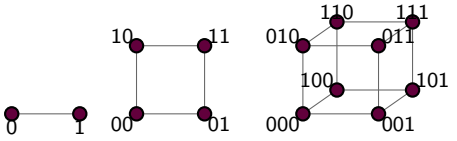


4-cube: two 3-cubes linked across dimension 3

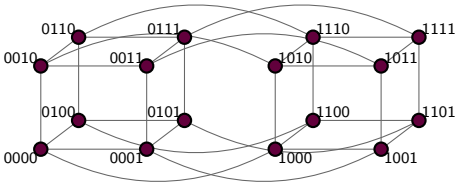
The hypercube

$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D? $d = \log_2 p$: flip the differing bits one by one.
- **Number of links** N_l? $p \log_2 p/2$: p nodes with d links each, every link counted twice.
- **Bisection width** L_B?



1-cube 2-cube 3-cube



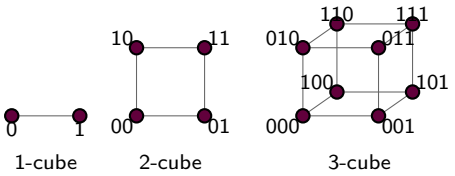
4-cube: two 3-cubes linked across dimension 3

The hypercube

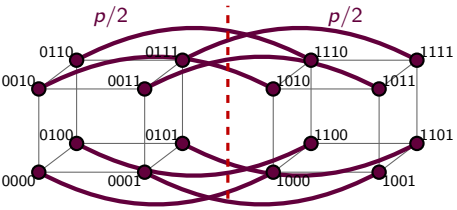
$p = 2^d$ processors, numbered in binary on d bits; two processors are linked when their numbers differ in exactly one bit. A d-cube is two $(d-1)$-cubes with the corresponding nodes linked.

- **Degree** k? $d = \log_2 p$: one link per bit.
- **Diameter** D? $d = \log_2 p$: flip the differing bits one by one.
- **Number of links** N_l? $p \log_2 p/2$: p nodes with d links each, every link counted twice.
- **Bisection width** L_B? $p/2$: the links of one dimension separate the two $(d-1)$-cubes: one link per node of a half.

Cutting the links of one dimension is **not the only way** to split the hypercube into two halves: can a cut that does not follow the dimensions remove fewer links? TD2.



1-cube 2-cube 3-cube



4-cube: two 3-cubes linked across dimension 3

Parameters of the classical topologies

Topology	Processors p	Degree k	Diameter D	Links N_l	Bisection L_B
Clique	p	$p-1$	1	$p(p-1)/2$	$(p/2)^2$
Ring	p	2	$\lfloor p/2 \rfloor$	p	2
Unidirectional ring	p	2 (1 in, 1 out)	$p-1$	p	2
2-D grid	$\sqrt{p} \times \sqrt{p}$	2 to 4	$2(\sqrt{p}-1)$	$2p-2\sqrt{p}$	$\sqrt{p}$
2-D torus	$\sqrt{p} \times \sqrt{p}$	4	$2\lfloor \sqrt{p}/2 \rfloor$	$2p$	$2\sqrt{p}$
Hypercube	$p = 2^d$	$d = \log_2 p$	$d = \log_2 p$	$p \log_2 p/2$	$p/2$

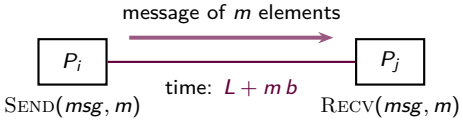
- In this crash course, the algorithm exercises focus **exclusively on the unidirectional ring** topology.

Communication cost

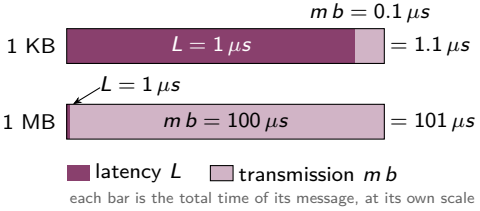
Sending a message of m elements over one link, from a processor to a neighbour, costs

$$L + \frac{m}{B} = L + m\,b$$

- L: the **latency**, the time to prepare and start the transmission, whatever the size of the message (in seconds).
- B: the **bandwidth** of the link, in elements per second; $b = 1/B$: the time to transmit one element.
- Two messages on the **same link** go one after the other; messages on **different links** travel at the same time.
- Small messages pay the latency, large ones the bandwidth: send **few large messages** rather than many small ones.



Example: $L = 1\,\mu s$, a 10 GB/s link, 8-byte elements: $b = 0.8$ ns; messages of 1 KB ($m = 125$) and 1 MB ($m = 125\,000$).



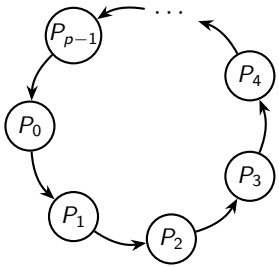
Outline

1. Distributed memory and communication networks
2. Network parameters and cost
3. Communication on a ring
4. Algorithms on a ring

Algorithms on a unidirectional ring

p processors $P_0, \ldots, P_{p-1}$; processor P_k has one link, to $P_{(k+1) \bmod p}$. Every processor runs the **same program**, with four routines:

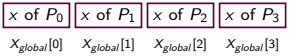
- MyRank(): returns the **rank** k of the processor, $0 \leq k < p$.
- NumProcs(): returns the number of processors p.
- Send(msg, m), called on P_k: sends m elements to P_{k+1}: a Send from P_k **goes to** P_{k+1}.
- Recv(msg, m), called on P_k: receives m elements from P_{k-1}: a Recv on P_k **expects a Send from** P_{k-1} (indices modulo p).
- msg is the **beginning of an array** of m elements, in the **local memory** of the processor: the elements sent, or the place where the received elements are stored. Arrays are always passed this way: **by name**, their size as a separate parameter.



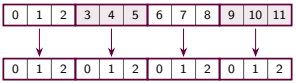
One program, many processes

- **Computation**: as in the PRAM, OpenMP and CUDA, we write a **single program** (one function); every process runs it with a **different value of** MyRank(), between 0 and $p-1$: the rank decides the branch to take (**if** $q = k \ldots$) and the elements to work on.
- The processes are **asynchronous**: at a given time, they can be at **different instructions** of the program; they only wait for each other at a Recv.
- **Data**: each process holds its own variable x: think of a **global array** $X_{global}[0 \ldots p-1]$, $X_{global}[i]$ being held by P_i in its own memory.
- Likewise, with an array $x[0 \ldots n-1]$ per process, P_i holds the i-th **block** of a global $X_{global}[0 \ldots N-1]$, $N = p\,n$: $X_{global}[i\,n+j]$ is $x[j]$ of P_i.
- Nobody holds the global array: it exists only in our head; using an element of another process takes a **message**.

a scalar x per process: $X_{global}[i] = x$ of P_i



an array $x[0 \ldots n-1]$ per process: the blocks of X_{global}

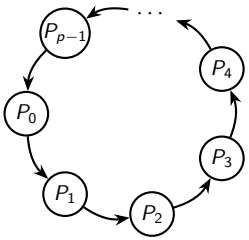


$X_{global}[0 \ldots N-1]$ with $N = p\,n$ elements:
$X_{global}[i\,n+j]$ is the $x[j]$ of P_i
here $p = 4$, $n = 3$, $N = 12$

Algorithms on a unidirectional ring (cont.)

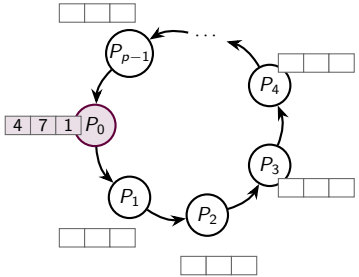
Rules of the game:

- SEND **does not block**: the processor moves on to the next instruction while the message travels in the background.
- RECV **blocks**: the processor cannot move on to the next instruction until the message has arrived; which makes sense, the next instructions may use the received data.
- A SEND on P_k must be matched by a RECV on P_{k+1}, a RECV on P_k by a SEND on P_{k-1}; otherwise a message is lost or a processor waits forever (**deadlock**).
- A processor can send and receive **at the same time**, on its two links: SEND$(\dots) \parallel$ RECV$(\dots)$.
- A message of m elements costs $L + m\,b$ on a link; messages on different links travel at the same time; an operation costs $\tau \ll L$.
- The **cost** of an algorithm: the time until the last processor is done.

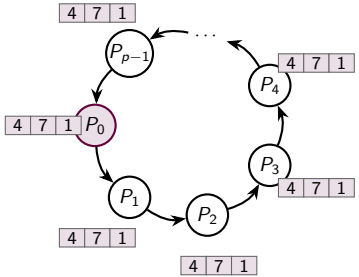


Broadcast on a ring

BCAST(k, msg, m): every processor has an array msg of m elements; the **root** P_k holds in its array the data to broadcast. At the end, the array of P_k has been **copied** into the array msg of every other processor. Example: $k = 0$, $m = 3$, $msg = [4, 7, 1]$ on P_0.



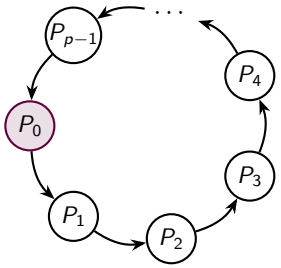
before: only the root P_0 holds data; the other arrays are empty



after: every processor holds a copy of $[4, 7, 1]$

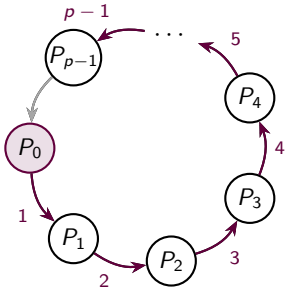
Broadcast on a ring: how?

- **Question**: P_k can only send to P_{k+1}, and every processor can only receive from its left neighbour. How can the data of P_k reach every processor? What does each processor have to do? How many steps?



Broadcast on a ring: how?

- **Question**: P_k can only send to P_{k+1}, and every processor can only receive from its left neighbour. How can the data of P_k reach every processor? What does each processor have to do? How many steps?
- **Idea**: relay the data around the ring, from each processor to the next one, every processor **keeping a copy** in its own array.
- Step 1: P_k sends to P_{k+1}; step 2: P_{k+1} sends to P_{k+2}; ...; step $p-1$: P_{k-2} sends to P_{k-1} (indices modulo p).
- The last link, from P_{k-1} back to P_k, is not used.



root $k = 0$: the data cross the link $P_{i-1} \to P_i$ at step i

Broadcast on a ring (cont.)

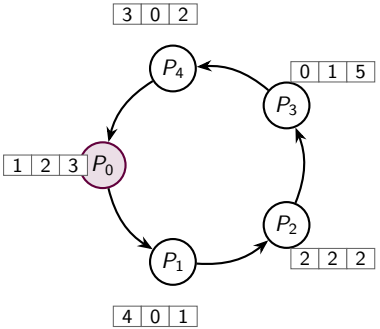
- The root performs one SEND and no RECV.
- The processor before the root, $P_{(k-1) \bmod p}$, performs one RECV and no SEND.
- All the others perform a RECV followed by a SEND.
- **Cost**: $p-1$ messages of size m, one after the other: $(p-1)(L + m\,b)$.
- Only **one link is busy at a time**: the other $p-1$ links are idle. Can we use the whole ring and do better? TD2.

BCAST(k, msg, m)

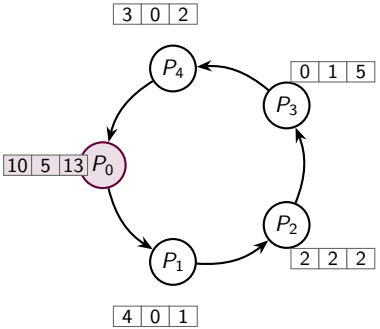
```
1: q ← MyRank(); p ← NumProcs()
2: if q = k then
3:     Send(msg, m)
4: else if q = (k − 1) mod p then
5:     Recv(msg, m)
6: else
7:     Recv(msg, m)
8:     Send(msg, m)
```

Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array msg; at the end, the **root** P_k holds in its array msg their **element-wise combination** with the associative and commutative operation $\otimes$ (sum, max, ...). The result **overwrites** the array of P_k; the arrays of the other processors **must not be modified**. Example: $k = 0$, $m = 3$, $\otimes = +$, $p = 5$.



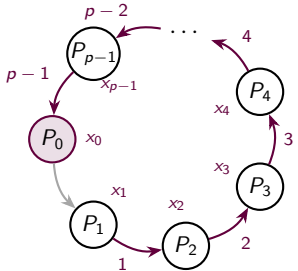
before: every processor holds its own msg



after: P_0 holds the element-wise sums; the others are unchanged

Reduction on a ring (cont.)

- The mirror image of BCAST: a partial result travels around the ring from P_{k+1} to P_k; every processor **combines** what it receives with its own elements and forwards the result.
- P_{k+1}: one SEND, no RECV; P_k: one RECV, no SEND, the last combination; the others: RECV, combine, SEND.
- Not given here: an exercise of TD2, with the structure of BCAST plus the combination.
- **Cost**: communication $(p-1)(L + m\,b)$, as for BCAST; computation $(p-1)\,m\,\tau$: $p-1$ combinations of m elements.
- A single link busy at a time again: the **pipelined** versions of BCAST and REDUCE, with better costs, are in TD2.



root $k = 0$, P_i holds x_i; at step s the link $P_s \to P_{s+1}$ carries $x_1 \otimes \cdots \otimes x_s$; P_0 combines x_0 last

Outline

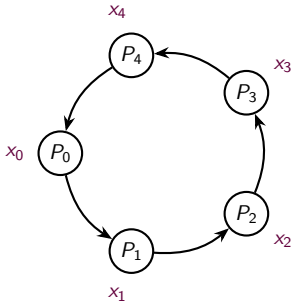
1. Distributed memory and communication networks
2. Network parameters and cost
3. Communication on a ring
4. Algorithms on a ring

A first algorithm: 1-D convolution

A vector $x[0 \dots p-1]$ is **distributed** over the ring: P_i holds the element $x[i]$ in its local variable x. We want to replace every element by the average of its neighbourhood:

$$x[i] \leftarrow \frac{x[i-1] + x[i] + x[i+1]}{3.0} \quad \text{for all } i, \text{ indices modulo } p.$$

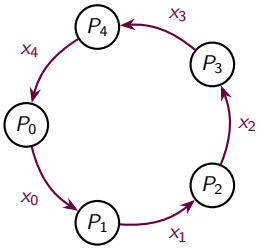
- A **1-D convolution** (a smoothing filter): the new value of $x[i]$ depends on $x[i-1]$ and $x[i+1]$, which live on the two neighbours of P_i.
- The vector is circular (indices modulo p), like the ring: $x[-1] = x[p-1]$ and $x[p] = x[0]$.
- All the new values use the **old** values: every processor must get its two neighbours' values before updating its own.
- P_i can only **receive from** P_{i-1} and **send to** P_{i+1}: how does it get $x[i-1]$? And $x[i+1]$?



the distributed vector, $p = 5$

Step 1: the left neighbour

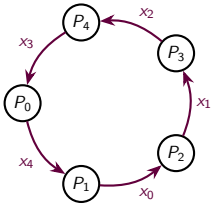
- $x[i-1]$ is on P_{i-1}, at the other end of the incoming link of P_i: **one step** is enough.
- Every processor sends its own value to the right and receives its left neighbour's value: $\textsc{Send}(x, 1) \parallel \textsc{Recv}(\mathit{left}, 1)$.
- All the links are used at the same time: one step of cost $L + b$ ($m = 1$ element).
- After step 1, P_i holds $x[i]$ and $x[i-1]$.



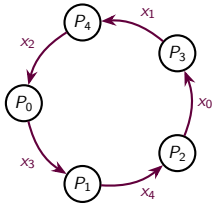
step 1: P_i sends x_i and receives x_{i-1}

Steps 2 to $p-1$: the right neighbour

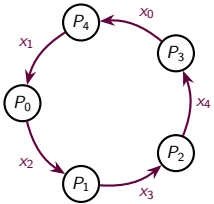
- $x[i+1]$ is on P_{i+1}, which can only send to the **right**: the value has to go **all the way round** the ring to reach P_i. After step 1, $x[i+1]$ is already on P_{i+2}; it still has $p-2$ links to cross: $P_{i+2} \to P_{i+3} \to \cdots \to P_{i-1} \to P_i$.
- At every step, every processor **forwards** the value it received at the previous step, and receives a new one: after step s, P_i has received $x[i-s]$; at step $p-1$, it receives $x[i-(p-1)] = x[i+1]$.
- $p-2$ more steps of cost $L+b$, all the links busy at every step.



step 2: P_i receives x_{i-2}



step 3: P_i receives x_{i-3}



step $p-1=4$: P_i receives $x_{i-4} = x_{i+1}$

1-D convolution: the algorithm and its cost

- Lines 2–3: step 1, $x[q-1]$ into *left*. Lines 4–6: steps 2 to $p-1$, forwarding the value received at the previous step, **copied** first into the send buffer so that it is not overwritten by the next one. Line 7: the last value received is $x[q+1]$.
- **Communication**: $p-1$ steps, one message of one element on every link: $T_{comm} = (p-1)(L+b)$.
- **Computation**: 3 operations: $T_{comp} = 3\tau$, negligible.
- The price of the **unidirectional** ring: a value from the right goes all the way round; bidirectional links would need one step.
- One element per processor: the $p-1$ messages cost far more than the sequential algorithm, $T_{seq} = O(p\tau)$: **slower** than sequential. The messages must be paid for by more **local work**: next example.

CONV1D(x)

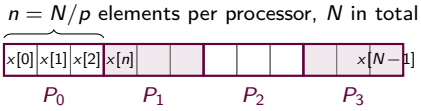
1: $q \leftarrow$ MYRANK(); $p \leftarrow$ NUMPROCS()
2: SEND(x, 1) ‖ RECV($tempR$, 1) ▶ step 1
3: $left \leftarrow tempR$ ▶ $x[q-1]$
4: **for** $i = 2, \ldots, p-1$ **do**
5: $tempS \leftarrow tempR$
6: SEND($tempS$, 1) ‖ RECV($tempR$, 1)
7: $right \leftarrow tempR$ ▶ $x[q+1]$
8: $x \leftarrow (left + x + right)/3.0$

Normalizing a vector

A vector $x[0 \dots N-1]$ of N elements, distributed by **blocks**: P_i holds the $n = N/p$ elements $x[in], \dots, x[(i+1)n-1]$ in a local array $x[0 \dots n-1]$ (N is divisible by p). We want to divide every element by the **2-norm** of the vector:

$$x[j] \leftarrow \frac{x[j]}{\|x\|_2}, \qquad \|x\|_2 = \sqrt{x[0]^2 + x[1]^2 + \cdots + x[N-1]^2}.$$

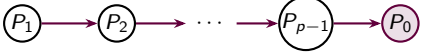
- NORMALIZE(x, n): called by every processor on its own block $x[0 \dots n-1]$; at the end, every element of every block has been divided by the norm of the **whole** vector.
- The new value of every element depends on **all** the N elements: a global quantity, the sum of the squares, to which every processor contributes and which every processor needs.



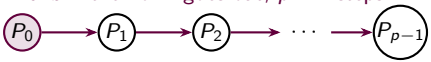
Normalizing a vector: the idea

- Step 1: every processor computes the sum of the squares of its **own block**, $s = \sum_{j=0}^{n-1} x[j]^2$: no communication.
- Step 2: the p partial sums are added up on one processor: a **Reduce** with $+$ into the root P_0, which obtains $\sum_{j=0}^{N-1} x[j]^2$.
- Step 3: P_0 takes the square root: it holds $\|x\|_2$.
- Step 4: every processor needs the norm: a **Bcast** from P_0.
- Step 5: every processor divides its own block by the norm: no communication.
- Exactly the job of the two routines: REDUCE brings the contributions in, BCAST sends the result out.

REDUCE: the sums of squares go in, $p-1$ steps



BCAST: the norm goes out, $p-1$ steps



Normalizing a vector: the algorithm and its cost

- Lines 3–4: $2n$ operations on the local block (a multiplication and an addition per element); lines 8–9: n divisions; line 5: a REDUCE of one element; line 6: a square root; line 7: a BCAST of one element.
- **Communication cost**: two trips around the ring with a single element: $T_{comm} = 2(p-1)(L+b)$.
- **Computation cost**: $T_{comp} = 3n\tau + (p-1)\tau + \tau$: the two local loops, the $p-1$ additions of the reduction and the square root; $T_{comp} \approx 3n\tau$ for large n.
- $T = T_{comm} + T_{comp}$; sequential: $T_{seq} = 3N\tau$. The speedup T_{seq}/T is close to p when $T_{comp} = 3(N/p)\tau$ dominates T_{comm}: for N large enough. The messages are the price of parallelism, the same whatever N.

NORMALIZE(x, n)

1: $q \leftarrow$ MYRANK()
2: $s \leftarrow 0$
3: **for** $j = 0, \ldots, n-1$ **do**
4: $\quad s \leftarrow s + x[j] \cdot x[j]$ ▶ local sum of squares
5: REDUCE$(0, s, 1, +)$ ▶ P_0: $s = \sum_{j=0}^{N-1} x[j]^2$
6: **if** $q = 0$ **then** $s \leftarrow \sqrt{s}$ ▶ the norm
7: BCAST$(0, s, 1)$ ▶ all: $s = \|x\|_2$
8: **for** $j = 0, \ldots, n-1$ **do**
9: $\quad x[j] \leftarrow x[j]/s$

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Paris, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com